Media Engineering and Technology Faculty
German University in Cairo

The Logic Advocate Tutor

Bachelor Thesis

Author: Adel Ashraf Mohamed Kamel Emam
Supervisors: Dr. Ahmed M. H. ABDELFATTAH
Submission Date: 8 June 2026

Media Engineering and Technology Faculty
German University in Cairo

The Logic Advocate Tutor

Bachelor Thesis

Author:	Adel Ashraf Mohamed Kamel Emam
Supervisors:	Dr. Ahmed M. H. ABDELFATTAH
Submission Date:	8 June 2026

This is to certify that:

(i) the thesis comprises only my original work toward the Bachelor's Degree

(ii) due acknowledgment has been made in the text to all other material used

Adel Ashraf Mohamed Kamel Emam
8 June 2026

Acknowledgments

I thank...

Abstract

Critical thinking is one of the things higher education is supposed to teach, but the tools we have to practise it on a screen all suffer from one of two opposite problems. Generative Large Language Models can hold a fluent debate on almost any topic, yet they exhibit *rhetorical rigidity*: once a model is assigned a stance, it tends to defend flawed reasoning with confidence rather than admit that a premise has been defeated. Formal argumentation frameworks have the opposite weakness. They can decide who is winning a debate with mathematical certainty, but they offer no natural conversational interface, and they reduce a learner's feedback to a flat accept-or-reject verdict. Neither half is a complete tutor on its own.

This thesis introduces the *Logic Advocate Tutor*, an interactive web application that brings the two halves together. The idea is to let the language model write the prose while a separate, deterministic engine writes the verdict. The engine is a custom Python evaluator that extends Dung's abstract argumentation framework with a parallel support relation, continuous acceptability scores in $[0, 1]$ in place of binary pass/fail labels, integer per-argument weights, and four value tags (Fact, Logic, Ethics, Emotion) borrowed from Bench-Capon's value-based audience model. On classical inputs the evaluator reduces to a thresholded grounded extension. It only departs from that binary world when bipolar, weighted, or value-tagged inputs ask for something more nuanced.

The conversational half runs on LLaMA-3 70B through the Groq inference endpoint. The model talks back through a strict pipe-separated response protocol and is never allowed to decide who is winning the debate; every score on the screen comes from the engine, not from the LLM. So the AI cannot pretend it is winning when the math says it has lost. When the engine declares the AI's argument defeated, the AI emits a clean concession sentinel and the debate ends. Voice input is handled by OpenAI's Whisper, image evidence by the LLaMA-3.2 90B Vision model, and online multiplayer by an ngrok tunnel and a QR code that lets two students on different networks share a debate in seconds. The whole thing runs inside Streamlit and stays within Groq's free tier, so a student can use it without a credit card.

A formal controlled user study was outside the time budget of a one-semester Bachelor project. I instead validated the system through structural inspection of the engine, informal play sessions with classmates, and constant observation while building it. The engine appeared to behave the way the formula predicts. The AI Layer stayed consistent with the engine's verdict in every session I watched. Latency was low enough that nobody complained. And in the informal feedback I collected, the moment the AI con-

ceded was the single most-cited reaction. A proper study with a baseline condition and a learning-outcome measurement is identified as the highest-priority follow-up work. The thesis itself is offered as evidence that the right way to build educational AI in the era of large language models is to keep the model and the rules separate, and to let the rules win whenever they disagree.

Contents

List of Figures

Chapter 1

Introduction

1.1 The Illusion of Competence in Educational AI

Bringing Artificial Intelligence (AI) into education has transformed several aspects of how we learn. In the past, automated tutors worked exceptionally well for objective subjects like math or programming. In those fields, there is a clear right or wrong answer, and student responses can be graded using strict rule based systems. But these traditional tutors often hit a wall when they try to support subjective decision making tasks [**?**]. In subjects that require critical thinking, ethics, or nuanced debate, conclusions are rarely simple. They depend on context, on weighing conflicting perspectives, and on understanding human values [**?**].

When we deal with these grey areas, we naturally rely on argumentation. This is a back and forth dialogue grounded in defeasible reasoning [**?**]. Defeasible reasoning means our conclusions are drawn from premises that are presumed but not deductively guaranteed. If a premise is successfully challenged by new and stronger evidence, the argument is defeated. Modern education leans heavily on this interactive process to develop students' critical thinking skills.

While human to human debate remains the best way to practise these skills, Large Language Models (LLMs) have emerged as powerful educational tools. In subjective tasks, an AI agent can act as a tireless devil's advocate [**?**]. It can offer different perspectives to stimulate a student's reasoning and expose them to viewpoints they might have overlooked. But capturing the fluid nature of human debate inside a computer system requires a rigid structural backbone. In computer science, we get that backbone from Computational Argumentation. The field is rooted in Dung's Abstract Argumentation Framework, which gives us a mathematical way to determine whether an argument survives based on how it attacks and defends other arguments [**?**].

1.2 The Problem of Rhetorical Rigidity

Despite the educational promise of AI, a serious problem shows up the moment we try to use off the shelf language models as standalone logic tutors: they suffer from an illusion of competence. Generative models are very good at producing human sounding text, but they often fail to capture the pragmatic reasoning required to surface implicit premises [**?**]. Those unstated assumptions are what actually make an argument persuasive in the first place.

Worse, when these models are tasked with arguing a specific viewpoint, they are highly susceptible to *rhetorical rigidity* [**?**]. By rhetorical rigidity I mean the tendency of an LLM, once assigned a stance, to keep making confident assertions even after the underlying logic has been defeated. Research shows that when an AI is forced to defend a rigidly assigned position, it tends to overcommit to flawed reasoning [**?**]. It adopts an artificially assertive tone instead of objectively evaluating the logical weight of a user's counter argument. In a classroom setting, this is disastrous. A student might present a perfectly valid and game winning logical point, only for the AI to hallucinate facts or stubbornly double down on a defeated premise. The lesson the student learns is that volume and stubbornness outweigh logical soundness, which is the opposite of what a tutor should be teaching.

Traditional logic tutors crafted by human experts go in the other direction. They are mathematically flawless but rigid and time consuming to build. They struggle to adapt to the messy and open ended nature of natural language. They cannot easily parse the subtle assumptions human debaters rely on. So we are left with a real gap in the current landscape: formal logic solvers that are mathematically accurate but conversationally dead, alongside modern language models that are conversationally brilliant but logically unreliable.

1.3 Constraining the Ghost: The Logic Advocate Tutor

To bridge this gap, this thesis introduces the **Logic Advocate Tutor**. It is a real time interactive tutoring system engineered to help students practise structured and logically grounded argumentation. The system is built on a novel premise: it synthesises the conversational fluidity of state of the art generative AI with the mathematical chains of Computational Argumentation. In essence, it constrains the language model inside a strict mathematical graph.

At its core, the system's logic engine upgrades Dung's standard Abstract Argumentation Framework to support Bipolar Argumentation and Gradual Evaluation Semantics. Instead of letting the AI guess who is winning the debate, the tutor translates every natural language input into a visual, directed graph of supports and attacks. A continuous

acceptability score in the range $[0.0, 1.0]$ is computed for every argument on the fly. The state of that mathematical graph is then injected directly into the system prompt, forcing the AI to stay consistent with the objective state of the debate. The model cannot claim victory if the math says its argument has been defeated.

To stay accessible and responsive in real time, the system uses the zero cost, high speed Groq API to power Meta's LLaMA 3 models, paired with OpenAI's Whisper for voice transcription. It also implements robust session management, including a custom manual save database and dynamic visual logic maps, which together create a persistent educational arena.

1.4 Thesis Contributions

The primary contributions of this project span both theoretical implementation and practical software engineering. They are:

1. **A custom Bipolar Gradual logic engine.** I designed and implemented a Python evaluator that translates natural language arguments into a weighted bipolar argumentation framework. The engine combines bipolarity, gradual semantics, value based weighting, and preference based normalisation, and computes continuous acceptability scores in $[0.0, 1.0]$.

2. **A protocol for grounding the LLM in the engine's verdict.** Using a structured pipe separated response format and context injection, the LLM is forbidden by construction from computing the verdict itself. Every score on the screen comes from the deterministic engine. This eliminates hallucination about debate outcomes and mitigates the rhetorical rigidity problem identified by recent work on multi agent LLM debate [**?**].

3. **An on demand automated hint system.** Drawing inspiration from data driven hint factory methodologies [**?**], the tutor surfaces a one sentence strategic hint targeted at the most recent enemy argument whenever the learner asks for one. The hint does not consume the learner's turn, which keeps the help non evaluative.

4. **Interactive visualisation and persistence.** The system integrates Graphviz to render real time, colour coded maps of the ongoing debate, alongside a horizontal momentum bar that summarises which side is currently ahead. A custom JSON based save format lets students preserve, load, and review past debates.

5. **A multimodal, classroom deployable architecture.** The whole system is delivered as a low latency Streamlit web application, with Whisper powered voice input, image evidence handled by the LLaMA 3.2 90B Vision model, and ngrok based QR code multiplayer joining for online debates between students on different networks.

The system's behaviour is examined informally in this thesis through structural inspection of the engine and small scale play sessions with classmates. A controlled study with a baseline condition and learning outcome measurement is identified in Chapter **??** as the highest priority follow up work.

1.5 Roadmap of the Thesis

This thesis traces the development of the Logic Advocate Tutor from its theoretical roots to its final technical execution.

Chapter 2 lays out the background and the theoretical groundwork. It covers Dung's Abstract Argumentation Framework, the evolution of bipolar and gradual semantics, the formal dialogue games that translate static graphs into interactive conversations, and the current limitations of language models in subjective decision making.

Chapter 3 describes the methodology. It walks through the four layer architecture of the system, the formal definition of the Bipolar Gradual logic engine, the dialogue game protocol that governs every move, and the pedagogical affordances that bring the engine in front of a learner.

Chapter 4 dives into the implementation. It walks through the Python that realises the design, paying close attention to the prompt engineering protocol that disciplines the LLM, the polled JSON architecture of the multiplayer subsystem, and the engineering trade offs I had to make along the way.

Chapter 5 looks at how the system actually performs. Within the time budget of a one semester project, no formal user study was conducted instead, the chapter reports structural observations about the engine, informal feedback from classmates who used the tutor, and a discussion of what a proper evaluation would require.

Chapter 6 concludes the thesis. It summarises the contributions, acknowledges the system's limitations, and lays out a concrete forward roadmap of future work in educational AI tools.

Chapter 2

Background and Literature Review

2.1 Introduction

The Logic Advocate Tutor does not exist in a vacuum. It sits at the intersection of three fields: formal computational logic, modern artificial intelligence, and educational pedagogy. To understand how the system works, we must trace how computer scientists translated human debate into mathematical graphs, how Large Language Models (LLMs) interact with those graphs, and how intelligent tutoring systems leverage data to keep students engaged.

2.2 The Bedrock of Logic: Abstract Argumentation Frameworks

The mathematical foundation of modern computational argumentation was established by Phan Minh Dung in his seminal 1995 paper [**?**]. Prior to Dung, evaluating an argument required parsing its internal text and logical proofs. Dung revolutionized the field by introducing the Abstract Argumentation Framework (AF), which shifted the focus to the macroscopic relationships between arguments instead [**?**].

In Dung's framework, arguments are treated as abstract nodes, and the focus is placed entirely on the binary "attack" relationships between them [**?**]. In this calculus of opposition, an argument's survival is not based on its inherent truth, but on its ability to defend itself against counter-arguments. To determine which arguments "win" a debate, Dung introduced acceptability semantics (such as the Grounded Extension), which mathematically calculate the set of arguments that remain undefeated after all attacks are resolved [**?**]. Subsequent work has consolidated and extended these semantics into a broad family. The modern handbook chapter by Baroni, Caminada, and Giacomin [**?**] provides a useful starting point for readers seeking a comprehensive view of the formal landscape.

2.3 Beyond Binary: Bipolar and Gradual Frameworks

While Dung's original framework excels at modeling negative interactions, human debate is rarely purely adversarial. Participants build upon each other's points, agree, and strengthen competing claims. To capture this reality, researchers introduced Bipolar Argumentation Frameworks (BAFs), which add a secondary "support" relation to the graph [**?**]. In a BAF, an argument can directly reinforce the acceptability of its target, simulating a more collaborative debate environment [**?**].

Traditional logic also evaluates arguments in a strict binary fashion: an argument is either accepted (`IN`) or defeated (`OUT`) [**?**]. Modern approaches, such as Gradual Evaluation Semantics, assign continuous acceptability scores in $[0.0, 1.0]$ [**?**]. Under gradual semantics, an argument is not destroyed by a single attack. Instead, its logical weight is proportionally diminished [**?**]. Besnard and Hunter's h-categorizer is a concrete and influential instantiation of this idea, scoring an argument by $1/(1 + \sum_{b \to a} s(b))$ [**?**], and several recent gradual semantics can be read as generalizations of it. Value-Based and Preference-Based Argumentation Frameworks expand on this theme by assigning numeric weights to specific categories of arguments, ensuring the math accounts for the fact that a weaker argument cannot freely defeat a stronger one [**?**].

2.4 Structuring the Conversation: Dialogue Games

To translate these static mathematical graphs into interactive chat applications, computer science utilizes "Dialogue Games" [**?**]. Dialogue games are rule-governed interactions in which two or more parties take turns making specific utterances, such as claims, challenges, concessions, or retractions [**?**]. The goal is to persuade the opponent or resolve a conflict of opinion.

When a conversation is forced to follow these rules, the interaction implicitly builds a logical graph of arguments and counter-arguments over time [**?**]. The integration of formal dialogue structures with natural language processing has paved the way for Argumentation-Based Chatbots [**?**]. Where standard chatbots rely on simple pattern matching or pure generative guessing, argumentation-based agents map user inputs to a formal knowledge base, allowing them to engage in more strategic persuasion and deliberation. Castagna et al. [**?**] survey several deployed systems in this family, including persuasion dialogue systems for medical decision-making and computational mediators that propose compromises in multi-party negotiation. The common thread across all of them is the same separation of structure (the formal graph) from surface (the generated text) that this thesis adopts.

2.5 The AI Frontier: LLMs and Subjective Decision-Making

The advent of Large Language Models has dramatically changed how conversational agents operate. In subjective decision-making tasks, where there is no single objective "ground truth," LLMs are increasingly used to present users with divergent rationales and to act as a collaborative "devil's advocate" [**?**].

However, relying entirely on LLMs in these environments is risky. In isolation, LLMs frequently struggle to grasp "implicit premises", the unstated, pragmatic assumptions that bridge a claim and its reasoning [**?**]. When an LLM is assigned a strict adversarial stance in a debate, it often exhibits "rhetorical rigidity" [**?**]. Instead of objectively evaluating the logic of an opponent's argument, the model overcommits to its assigned stance, defending flawed reasoning with unwarranted confidence and extreme tones [**?**]. To address this, researchers have suggested that embedding a computational argumentation engine inside the generative loop can mitigate these weaknesses by anchoring the model's text generation to a verifiable, mathematical state [**?**].

The literature so far diagnoses the rhetorical-rigidity problem and gestures at this kind of fix at the conceptual level. What this thesis adds is a concrete, deployable instance of that fix: an undergraduate-targeted tutor in which the LLM is genuinely unable to compute the verdict of a debate, because that computation is wrested from it and handed to a separate engine.

2.6 Educational Technology and Automated Hint Generation

In the educational sector, argumentation is recognized as a fundamental instrument for developing critical thinking and collaborative problem-solving skills [**?**]. Building Intelligent Tutoring Systems (ITS) that can effectively guide students through complex logic problems is notoriously difficult, primarily because it traditionally requires immense time from subject-matter experts to hand-craft every possible rule and response [**?**].

To bypass this bottleneck, data-driven approaches like the "Hint Factory" have been developed to automate the generation of formative feedback [**?**]. By mapping out historical student interactions as Markov Decision Processes, these systems can automatically derive context-specific hints based on optimal solution paths [**?**]. Experimental evaluations in deductive logic courses show that students provided with on-demand, automated hints attempt and complete significantly more problems [**?**]. They are also substantially less likely to abandon the tutor out of frustration [**?**].

The Logic Advocate Tutor draws on the on-demand and non-evaluative principles of the Hint Factory [**?**], while implementing the hint generator itself as a single LLM call rather than as an MDP-based search over historical interactions. The technical mechanism is therefore different from Stamper et al.'s system, but the pedagogical philosophy (providing help when asked without penalizing the request) remains the same.

2.7 Summary

The literature reveals a clear trajectory. Formal argumentation provides flawless mathematical tracking of debates [**?**], and LLMs provide unparalleled conversational fluidity [**?**]. Yet neither is sufficient as a standalone educational tool. LLMs suffer from rhetorical rigidity [**?**], and abstract graphs lack natural human interaction. To keep students engaged, intelligent tutors also need dynamic, non-evaluative hint systems [**?**]. The Logic Advocate Tutor synthesizes these strands of research, using a Bipolar Gradual logic engine to constrain a generative LLM inside a formal dialogue game, thereby creating a robust, interactive environment for teaching argumentation.

Chapter 3

Methodology

3.1 Overview

This chapter breaks down how I designed the Logic Advocate Tutor before writing a single line of production code. It covers the abstractions I built to connect formal argumentation theory with a standard chat-style interface. I also explain why I chose this specific architecture over other available options. The next chapter will walk through the actual Python code that brings this design to life.

The core challenge boils down to one goal: building a debate environment where any natural-language argument is forced to follow the strict survival rules of a formal argumentation framework, while still feeling as responsive as a normal chat app. To make this happen, the system has to juggle four jobs at once. It must accept free text from a human or a Large Language Model, map those arguments into a directed graph of attacks and supports, compute a real-time acceptability score for every node, and display that score so players can adjust their strategy.

That is why I split the system into four cooperating layers. There is a **Presentation Layer** for the user interface, a **Dialogue Layer** that enforces the rules of the game, a **Logic Layer** that holds the mathematical framework, and an **AI Layer** that talks to the LLM. Figure **??** shows how these pieces fit together. The key design choice here is strict isolation. Each layer only knows about the one immediately below it. This means I can swap out the language model or completely redesign the UI without ever breaking the core math.

3.2 Design Goals and Non-Goals

Before diving into the architecture, I want to be clear about the specific problems this system tries to solve, as well as the problems I intentionally left out of scope. These goals directly address the gaps I identified back in Chapter **??**.

Goal 1: Keep the LLM honest. Standalone AI debate agents often suffer from rhetorical rigidity [**?**] and tend to claim victory no matter what actually happens in the debate [**?**]. So, the system needs its own ground truth. I built a deterministic algorithm to calculate the scores rather than relying on a model that might lie to sound confident. The LLM gets to write the text and rate the persuasiveness of an argument, but it never gets to decide who is winning.

Goal 2: Continuous scores over pass/fail labels. Classical Dung-style semantics label every argument as either `IN` or `OUT` [**?**]. That binary verdict works for theory, but it is practically useless for a student. If a learner's claim is simply labeled "defeated," they have no idea how close it was to surviving or whether adding one more supporting premise would have saved it. Following the gradual semantics literature [**?**, **?**], I assigned every argument a continuous score between 0.0 and 1.0. This lets students watch their argument get weaker gradually rather than facing an abrupt defeat.

Goal 3: Make support a first-class citizen. Real classroom debates are collaborative. People build on each other's points all the time. To reflect this, I extended the basic attack relation with a parallel *support* relation, adapting the methods of bipolar argumentation frameworks [**?**].

Goal 4: Different audiences value different arguments. A factual claim should hit harder against an emotional appeal than the other way around. Following Bench-Capon's value-based framework [**?**], I attached a value tag to every argument. This tag scales the mathematical strength of attacks and supports.

Goal 5: Run anywhere for free. The system needs to live inside a standard web browser, accept both text and voice, and let two students share a session without installing anything. It also needs to run entirely on the free tiers of its cloud services so students never hit a paywall.

What I am not trying to do. This system does not perform automatic argument mining on long essays [**?**]. It does not detect informal logical fallacies in the text, and it does not try to parse Toulmin-style warrants [**?**]. Just like Dung's original method [**?**], each argument is treated as a black box. The core contribution is how these nodes are scored, displayed, and constrained by the LLM, not how their internal grammar is parsed.

3.3 The Four-Layer Architecture

Figure **??** illustrates the layers and the flow of data. A user move enters at the top and pushes downwards. The freshly computed scores and the AI's reply then travel back up to the screen.

Presentation Layer. This is the part the user actually sees and interacts with. It captures inputs, draws the chat history as colour-coded bubbles, renders the live argumentation graph using Graphviz [**?**], and displays the running acceptability percentages.

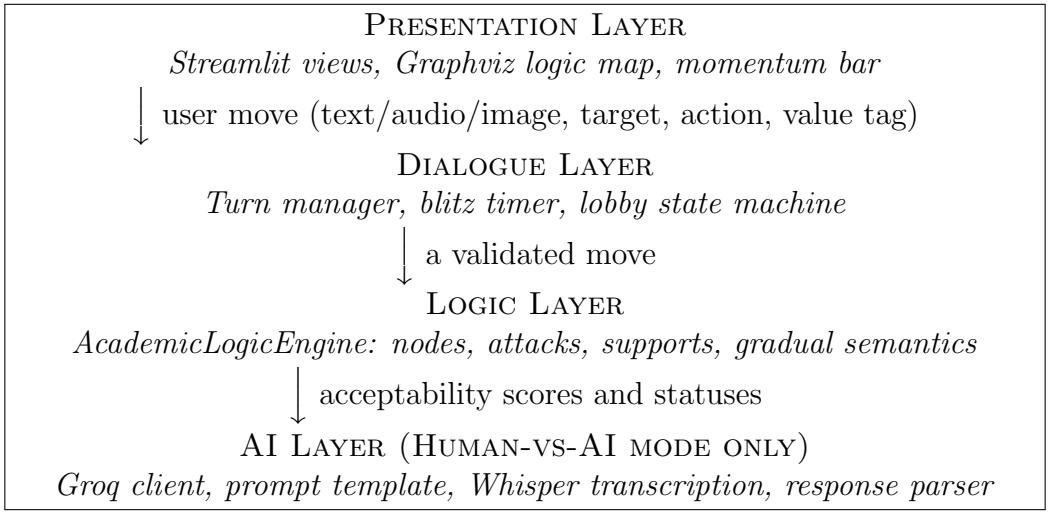


Figure 3.1: The four-layer architecture. Each layer interacts exclusively with the layer right below it.

Because Streamlit [**?**] reruns the entire script every time the user clicks a button, this layer has to be completely reconstructible from the session state alone.

Dialogue Layer. This layer enforces the rules of the debate game I lay out in Section **??**. It manages the turn order, an optional 60-second blitz timer, and the protocol that lets players officially end a debate. In single-machine mode, this lives inside Streamlit's session state. In online multiplayer, it runs through a lightweight JSON-backed lobby file (`lobby_db.py`) that both clients poll.

Logic Layer. This is where the formal argumentation actually happens. It exposes one main class called `AcademicLogicEngine`. This class takes in the arguments, attacks, and supports, and then spits out a vector of continuous acceptability scores. The engine knows absolutely nothing about the user interface, the LLM, or the network. You could rip it out of the project and unit test it completely on its own.

AI Layer. This layer only wakes up when the user selects Human-vs-AI mode. Its job is to read the chat transcript, write a counter-argument, score both the human's point and its own on a 1-to-25 scale, and pick a value tag. The output gets passed back up to the Dialogue Layer just like a normal human move.

3.4 The Bipolar Gradual Engine

The mathematical heart of the project is a custom evaluator that mashes together four major ideas from the literature. Bipolarity comes from Cayrol and Lagasquie-Schiex [**?**].

Gradual semantics comes from [**?**, **?**]. Value-based weighting is inspired by Bench-Capon [**?**]. Finally, preference-based normalization draws from Amgoud and Cayrol [**?**]. This section explains the math, while the next chapter will show the Python code that runs it.

3.4.1 Framework definition

A debate is a tuple

$$\mathcal{F} = \langle \mathcal{A}, \mathcal{R}_{\text{att}}, \mathcal{R}_{\text{sup}}, w, v \rangle,$$

where $\mathcal{A}$ is a finite set of arguments, $\mathcal{R}_{\text{att}} \subseteq \mathcal{A} \times \mathcal{A}$ is an attack relation, $\mathcal{R}_{\text{sup}} \subseteq \mathcal{A} \times \mathcal{A}$ is a support relation, $w : \mathcal{A} \to \{1, 2, \ldots, 25\}$ assigns each argument an integer weight, and $v : \mathcal{A} \to \mathcal{V}$ tags each argument with one of four value labels:

$$\mathcal{V} = \{\textsc{Fact}, \textsc{Logic}, \textsc{Ethics}, \textsc{Emotion}\}.$$

I hand-picked a multiplier for each value tag, inspired by Bench-Capon's audience model [**?**]:

$$\mu(\textsc{Fact}) = 1.2, \quad \mu(\textsc{Logic}) = 1.1, \quad \mu(\textsc{Ethics}) = 1.0, \quad \mu(\textsc{Emotion}) = 0.8.$$

3.4.2 The acceptability score

For every argument $a \in \mathcal{A}$ the engine computes an acceptability score $s(a) \in [0.0, 1.0]$. The score is the fixed point of the following update rule, computed by ten rounds of synchronous relaxation starting from $s^{(0)}(a) = 1.0$ for every argument:

$$\alpha(a) = \sum_{(b,a) \in \mathcal{R}_{\text{att}}} s(b) \cdot w(b) \cdot \frac{\mu(v(b))}{\mu(v(a))},$$

$$\sigma(a) = \sum_{(b,a) \in \mathcal{R}_{\text{sup}}} s(b) \cdot w(b) \cdot \frac{\mu(v(b))}{\mu(v(a))},$$

$$s^{(t+1)}(a) = \min\left(1.0, \frac{1.0 + \sigma(a)/\max(1, w(a))}{1.0 + \alpha(a)/\max(1, w(a))}\right).$$

Once the loop converges, I extract a coarse binary status label so the UI has something simple to render:

$$\text{status}(a) = \begin{cases} \texttt{IN} & \text{if } s(a) \geq 0.5, \\ \texttt{OUT} & \text{otherwise.} \end{cases}$$

3.4.3 Why this formula

The fraction shape $\frac{1+\sigma}{1+\alpha}$ borrows its logic from the h-categorizer [**?**], which divides initial strength by attack strength. I generalized it for the bipolar case by adding a matching numerator for support. Three things make this the perfect fit for a tutoring system. First, the score is strictly bounded. Because both the top and bottom grow linearly, the score never escapes $[0.0, 1.0]$ once capped. This keeps the colour gradients in the UI predictable. Second, the score is sensitive to weight. A stronger argument is mathematically harder to take down, which gives me the preference-based behaviour I wanted from [**?**]. Third, the score reacts beautifully to value tags. An emotional appeal landing on a factual claim mathematically registers as weaker than the reverse, exactly as Bench-Capon predicts [**?**].

3.4.4 Relationship to classical Dung

If you turn off the support relation ($\mathcal{R}_{\text{sup}} = \varnothing$), set every weight to 1, and only use one value tag, my gradual rule collapses into the standard h-categorizer [**?**]. In that isolated scenario, an unattacked argument stays at 1.0, while an argument hit by a fully accepted attacker drops to 0.5. If you threshold at 0.5, the cascade approximates Dung's grounded extension [**?**]. So on standard inputs, my engine behaves just like a classical evaluator. It only departs from the classics when bipolar, weighted, or valued parameters require a more nuanced calculation.

3.4.5 From the prototype to the deployed engine

The repository still holds my early prototype in `class Argument.txt`. That version used a strict preference rule: an attack only counted if the attacker's weight was equal to or greater than the target's. Otherwise, the attack was silently ignored, following Amgoud and Cayrol [**?**]. While that looked clean on paper, it was incredibly brittle in practice. The LLM would occasionally deploy a rhetorically clever but low-weight attack against a heavyweight claim, and the prototype would just reject the move entirely. From a pedagogical standpoint, this was awful because the disagreement vanished from the screen. So, I softened the rule. In the deployed engine, a weaker attacker still chips away at the target a little bit, but cannot force its score below $1/(1+\alpha)$. This ensures the learner actually sees the attack land and can judge its severity for themselves.

3.5 The Dialogue Game

The flow of moves follows a simplified version of the dialogue games described by Prakken [**?**]. A standard debate goes through three phases.

Opening. Side A always speaks first. Their opening move is the *Main Claim* and it needs no target since the graph is empty. They just pick a value tag and post the argument.

Argumentation. For every turn after that, the active player picks three things: an action (*Attack* or *Support*), a target node that already exists, and a value tag for their new argument. The UI automatically filters the targets: attacks must point at the opponent, while supports must reinforce the player's own nodes. After the new node is added, the engine recalculates all the scores and passes the floor. In Human-vs-AI mode, the LLM fires back immediately. In multiplayer modes, control alternates between the humans.

Closing. A debate ends in one of three ways: someone hits the "End Debate Now" button, both players mutually agree to end the session online, or the LLM formally gives up by outputting the `CONCEDE` sentinel in Human-vs-AI mode. At the end, a victory banner shows the final status of the root claim. Side A wins if and only if their Main Claim remains `IN`.

Optional blitz mode. A toggle on the host dashboard activates a 60-second timer per turn. If the timer runs out, the slow side forfeits their turn. In Human-vs-AI mode, running out of time loses the whole debate since the AI operates instantly. I originally built this just to stress-test system latency, but it ended up being one of the most fun ways to play.

3.6 Move Validation

Before any input touches the Logic Layer, the Dialogue Layer enforces three strict validation rules:

1. Except for the Main Claim, every move must target a node that already exists in the graph.
2. An *Attack* edge must cross between opposing sides, and a *Support* edge must stay within the same side.
3. Every move must carry a valid tag from $\mathcal{V}$.

The first rule prevents fragmented graphs. The second rule stops players from attacking themselves to artificially deflate their own score and game the momentum bar. The third rule is mathematically necessary because the gradual semantics formula breaks if a tag is missing.

3.7 Multimodal Input

Three different input channels feed into the Dialogue Layer pipeline:

- **Text.** The default interface is a single-line text field styled like a modern chat app. A lightweight `keydown` listener binds the Enter key directly to the submit button, making it feel fast and frictionless.
- **Voice.** The composer has a microphone toggle to capture audio directly in the browser. The recording is routed to OpenAI's Whisper model [**?**] via Groq's inference endpoint, which transcribes the speech into text. I added this primarily so blitz mode would not unfairly punish slow typists.
- **Image Evidence.** An attachment button accepts image files. When an image is uploaded, it gets encoded and sent alongside the text prompt to the LLaMA-3.2 90B Vision model [**?**]. This lets the AI actually see and verify any visual evidence the student provides.

All three channels resolve into the exact same in-memory text representation before reaching the Logic Layer, keeping the formal framework totally agnostic to how the user submitted their move.

3.8 Multiplayer Topology

The system has three modes. *Human-vs-AI* and *Human-vs-Human (Local)* are single-machine instances. The first pits the student against the LLM, while the second lets two students pass a laptop back and forth. *Online Debate* introduces distributed play.

In online mode, the host machine creates a lobby secured by a unique room ID. This lobby is simply a lightweight JSON file (`lobbies.json`). Both clients check this file every three seconds using Streamlit's auto-refresh architecture. Because the lobby lives on the host's local disk, external players need network access. To solve this, the system automatically opens an ngrok tunnel [**?**] and generates a QR code containing the public URL using the `qrcode` library [**?**]. Students just scan the code with their phones to join instantly. Meanwhile, the host dashboard acts as an educator interface, showing the live graph, momentum metrics, and override controls.

I deliberately chose a polled JSON file over a real-time database like Firebase. The system needed to be ready for classroom deployment requiring nothing more than the host laptop. A three-second polling interval is plenty fast enough for a typed debate, and it keeps the technical dependencies incredibly low.

3.9 Pedagogical Affordances

This architecture is designed to be an educational scaffold, actively supporting the reflective practice described by Rapanta and Macagno [**?**] and Le [**?**]. Three specific features make this goal a reality:

The Interactive Logic Map. An expandable panel renders the current debate as a directed graph using Graphviz [**?**]. Nodes are filled with a colour gradient interpolating between red ($s = 0.0$), yellow ($s = 0.5$), and green ($s = 1.0$). Attack edges are red, and support edges are blue. This map gives the learner an immediate visual explanation for their logical standing, providing a mathematical ground truth that the AI cannot manipulate.

The Momentum Bar. A horizontal stacked bar above the chat displays the cumulative weight of each side's surviving arguments as a percentage. While the per-node scores show the health of individual arguments, the momentum bar gives a macro-level summary of the entire debate. Testers found this incredibly helpful for quick reference during rapid exchanges.

On-Demand Hints. A dedicated button prompts the LLM to generate a single-sentence strategic hint targeting the opponent's most recent argument. I kept the hint generation pipeline completely separate from the counter-argument generator so students can ask for help without losing their turn. This feature draws directly from the Hint Factory methodologies of Stamper et al. [**?**], which proved that providing on-demand, non-evaluative hints drastically reduces student frustration and dropout rates in logic tutors.

3.10 Conclusion

The architecture I just laid out places a deterministic, mathematically precise argumentation framework at its core, wrapping it in three layers that handle the conversation, the UI rendering, and the AI generation. The resulting framework is bipolar, weighted, value-tagged, and gradual. Crucially, it is the only component allowed to declare the objective state of the debate. The following chapter dives into the Python code that makes this design a reality and discusses the engineering trade-offs I had to make along the way.

Chapter 4

Implementation

4.1 Overview

This chapter breaks down the actual code that turns the design from Chapter ?? into a working application. I wrote everything in Python 3.11, and the user interface runs on Streamlit 1.32 [?]. The entire repository consists of seven Python files divided between a top-level package and a `views/` subfolder:

`app.py`	Entry point, mode router
`logic_engine.py`	The Logic Layer
`ai_agent.py`	The AI Layer (Groq client and prompts)
`database.py`	Legacy JSON persistence helper
`lobby_db.py`	Online-mode lobby store
`views/local_debate.py`	Single-machine UI (Human-vs-AI and Local)
`views/mobile_multiplayer.py`	Online host dashboard and player view

I will walk through these files in the exact order a debate move travels through the system. We will start with the Logic Layer because everything else depends on it, and finish with the multiplayer subsystem.

4.2 The Logic Engine

The Logic Layer lives entirely inside a single file called `logic_engine.py`. It contains one constant table and one main class. The constant table locks in the value-tag multipliers from Section ??:

```
VALUE_WEIGHTS = {
    "Fact":    1.2,
    "Logic":   1.1,
    "Ethics":  1.0,
    "Emotion": 0.8,
}
```

These four numbers are the most opinionated part of the entire system. I did not just pull them from a research paper. I hand-picked them so that a Fact-tagged attack hitting an Emotion-tagged claim would land with about 50% more weight than the reverse. This aligns perfectly with the rough asymmetry Bench-Capon describes in his worked examples [**?**], while still giving emotional appeals enough mathematical power to do real work in the debate.

4.2.1 The `AcademicLogicEngine` class

The main class is compact enough to show in its entirety:

```
class AcademicLogicEngine:
    def __init__(self):
        self.nodes    = {}   # mid -> {"text", "weight", "value_tag"}
        self.attacks  = []   # list of (attacker_id, target_id)
        self.supports = []   # list of (supporter_id, target_id)
        self.statuses = {}   # mid -> "IN" | "OUT"
        self.scores   = {}   # mid -> float in [0, 1]
```

The four state variables directly map to the formal tuple $\langle \mathcal{A}, \mathcal{R}_{\text{att}}, \mathcal{R}_{\text{sup}}, \dots \rangle$ defined in Section **??**. The `nodes` dictionary is keyed by a message ID like `"Msg_3"`. The two relations are just flat lists of pairs. I included three setter methods that are exactly one line each: `add_argument`, `add_direct_attack`, and `add_support`. Notice that none of them validate the input. I pushed all the validation up to the Dialogue Layer because I wanted this engine to remain a pure mathematical object. You can isolate it and run unit tests on it without needing any other part of the project.

4.2.2 The semantics loop

The entire algorithm from Section **??** fits inside a single method called `evaluate_semantics`:

```
def evaluate_semantics(self):
    self.scores = {mid: 1.0 for mid in self.nodes}
    for _ in range(10):                                    # ten
   relaxation steps
        new_scores = {}
        for mid in self.nodes:
            t_val = self.nodes[mid].get("value_tag", "Logic")
            attack_sum = 0
            for atk, tgt in self.attacks:
                if tgt == mid and atk in self.nodes:
                    a_val = self.nodes[atk].get("value_tag", "Logic")
                    mult  = VALUE_WEIGHTS[a_val] / VALUE_WEIGHTS[t_val]
                    attack_sum += self.scores[atk] * \
                                  self.nodes[atk]["weight"] * mult
            support_sum = 0
            for sup, tgt in self.supports:
                if tgt == mid and sup in self.nodes:
```

```
                    s_val = self.nodes[sup].get("value_tag", "Logic")
                    mult  = VALUE_WEIGHTS[s_val] / VALUE_WEIGHTS[t_val]
                    support_sum += self.scores[sup] * \
                                   self.nodes[sup]["weight"] * mult
            tw = max(1, self.nodes[mid]["weight"])
            score = (1.0 + support_sum / tw) / (1.0 + attack_sum / tw)
            new_scores[mid] = min(1.0, score)
        self.scores = new_scores
    for mid in self.nodes:
        self.statuses[mid] = "IN" if self.scores[mid] >= 0.5 else "OUT"
```

There are a few important details in this loop. First, it reads from `self.scores` and writes to a fresh dictionary called `new_scores`. This means every update inside a given iteration looks at the exact same previous state. I chose synchronous relaxation over the asynchronous version. Even though async converges faster, it makes the final result depend on the exact order the arguments were added. That creates a nightmare for reproducibility and debugging. The fixed loop count of ten is actually overkill for a standard debate graph. In practice, the score vector usually stops changing completely before the seventh iteration, which I discuss more in Chapter **??**. I skipped adding an explicit convergence test because the time spent looping is microscopic compared to the LLM network call anyway. The loop also safely ignores self-attacks and self-supports. The Dialogue Layer prevents those from happening in the first place, so ignoring them here saves me from writing a messy special case. Finally, I cap the final score at 1.0. Without that cap, a heavily supported argument with zero attacks could climb above 1.0, which would break the colour-rendering logic in the user interface.

4.2.3 A worked example

To see how this works in practice, imagine three arguments entering the graph with weights of 10, 8, and 5, tagged as Fact, Logic, and Emotion:

$$\begin{aligned} a_1 &: \text{“Cars cause smog.”} && w = 10,\ v = \textsc{Fact} \\ a_2 &: \text{“Electric vehicles do not.”} && w = 8,\ v = \textsc{Logic} \quad \text{supports } a_1 \\ a_3 &: \text{“But people like the engine sound.”} && w = 5,\ v = \textsc{Emotion} \quad \text{attacks } a_1 \end{aligned}$$

After ten iterations, the engine reports $s(a_1) \approx 0.91$, $s(a_2) = 1.0$, and $s(a_3) = 1.0$, leaving all three with an `IN` status. The factual claim easily survives because the emotional attack is handicapped by the $\mu(\textsc{Emotion})/\mu(\textsc{Fact}) = 0.8/1.2 = 0.667$ ratio, while the supporting logical argument restores $1.1/1.2 \approx 0.917$ of its own weight. If we swap the supporter to be an Emotion tag instead of a Logic tag, the score for a_1 drops to roughly 0.63. That is exactly the value-aware behaviour I wanted the framework to produce. Seeing those numbers actually fall out of the math for the first time was the moment I knew the formula was solid.

4.3 The AI Layer

The AI Layer lives in `ai_agent.py` and exposes three main functions: `transcribe_audio`, `generate_counter_argument`, and `generate_hint`. All three act as wrappers around the Groq Cloud inference API [?]. I chose Groq over OpenAI or Anthropic for two practical reasons. First, Groq's specialized hardware returns LLaMA-3 70B [?] responses in single-digit seconds. That keeps the perceived latency low enough that chatting feels natural. Second, Groq's free tier is generous enough that I never hit a rate limit during my thesis testing, meaning no student ever needs to enter a credit card to use the app.

4.3.1 Generating a counter-argument

`generate_counter_argument` is the heaviest lifter in the AI Layer. It grabs the full chat transcript, the latest user text, and any attached media, then returns five specific outputs. Here is the trimmed-down version:

```
def generate_counter_argument(messages, latest_text,
                              uploaded_file=None,
                              saved_media_path=None,
                              media_type=None):
    transcript = build_transcript(messages)
    prompt     = STYLE_PROMPT + transcript + INSTRUCTIONS.format(
                     latest=latest_text)
    model      = 'llama-3.3-70b-versatile'
    if uploaded_file and media_type.startswith("image"):
        model = 'llama-3.2-90b-vision-preview'
        msg = [build_text_block(prompt),
               build_image_block(saved_media_path, media_type)]
    else:
        msg = [{"role": "user", "content": prompt}]
    resp = client.chat.completions.create(
            model=model, messages=msg,
            temperature=0.7, max_tokens=1024)
    return parse(resp.choices[0].message.content)
```

The prompt itself is a massive format string written in the second person. It tackles three jobs simultaneously. First, it assigns the AI a role as a debate partner in a student project, and explicitly commands it to never refuse the topic. I added that strict instruction on my second day of testing because the model kept acting overly cautious and refusing to debate contentious topics. Adding that one sentence was the best fix I made to the entire prompt. Second, it enforces a specific style. The AI is told to use short, casual sentences, drop the heavy jargon, and ignore user typos. Without this rule, LLaMA-3 defaults to a dense, academic tone that sounds incredibly condescending in a chat window. Finally, it locks in the communication protocol. The response must be exactly four pipe-separated fields formatted as `HumanScore|AIScore|ValueTag|CounterText`. If the AI realizes the human just made an unanswerable point, it must output the exact string `0|0|Logic|CONCEDE`.

I admit the pipe protocol looks a bit ugly, but it is vastly easier to parse than JSON when the model occasionally hallucinates stray markdown formatting. The parser is built to be forgiving:

```
try:
    parts = response_text.split("|", 3)
    human_weight = max(1, min(25, int(parts[0].strip())))
    llm_weight   = max(1, min(25, int(parts[1].strip())))
    llm_val      = parts[2].strip()
    if llm_val not in {"Logic", "Fact", "Ethics", "Emotion"}:
        llm_val = "Logic"
    llm_text     = parts[3].strip()
except Exception:
    human_weight = llm_weight = 12
    llm_val      = "Logic"
    llm_text     = response_text
```

If the parsing fails, I just let it degrade gracefully. The raw output becomes the counter-argument text, and the weights default to a neutral 12. This is significantly better than crashing the app because the human still gets a reply, keeping the educational flow moving while only sacrificing a bit of score calibration.

4.3.2 How the LLM gets grounded

The most important pedagogical claim of this entire project is that the LLM cannot fake who is winning. Interestingly, this is not enforced by the prompt. It is enforced by the Python code wrapping it. After every human move, the engine recalculates the math. Those math scores dictate the momentum bar, the node colours, and the percentage labels. The LLM never even sees these scores. It only sees the raw text transcript. There is absolutely no way for the LLM to pretend a defeated argument is still alive because that judgment is executed by deterministic Python code the moment the move hits the graph. If a learner delivers a knockout argument, the engine will flag the AI's previous reply as `OUT` regardless of how confident the AI sounds. The user interface always renders the engine's verdict, never the LLM's.

4.3.3 Generating a hint

The `generate_hint` function is much simpler. It takes the opponent's most recent argument and asks the model for a very brief, single-sentence strategic tip on how to attack its logical weak points. The output is strictly capped at 100 tokens. To make it clear that this is advice and not a debate move, the hint renders inside a blue info box instead of a standard chat bubble. Requesting a hint also does not increment the turn counter, so a student can ask for help as many times as they want without losing their turn. During early testing, I tried treating the hint as an actual debate move, but putting it in a side channel felt far less punishing. This perfectly matches the design recommendations from Stamper et al. [**?**].

4.3.4 Voice transcription

The `transcribe_audio` function takes a recorded audio buffer and fires it off to Groq using the Whisper-large-v3 model, returning the transcribed text. The actual recording happens right in the browser using Streamlit's native audio input component. I save the audio buffer to an uploads folder with a timestamp. This allows an educator to go back and listen to what the student actually said, and it also avoids issues with Whisper sometimes rejecting in-memory streams in favor of hard file paths.

4.4 The Local Debate View

The `views/local_debate.py` file is the largest piece of the project, clocking in at around 470 lines. It handles the entire single-machine experience and relies entirely on Streamlit's session state to remember what happened between user clicks.

4.4.1 Initialisation

The first time the page loads, the view builds a fresh `AcademicLogicEngine` and stores it in the session state. All the other persistent variables like the message list, turn counters, and timers live there too. Since Streamlit reruns the whole script every time a user interacts with the app, anything that needs to survive a click has to be stashed in the session state. Normal global variables just get wiped out.

4.4.2 The composer bar

The composer is a clean three-column layout featuring an attachment button, a text input field, and a primary submit button. I injected a tiny JavaScript snippet to listen for the Enter key and map it to the submit button. This is literally the only piece of imperative JavaScript in the whole repository. I had to add it because Streamlit's default Enter-key behaviour misses the stop-recording event when the microphone is active.

4.4.3 The thinking animation

When a student submits a move against the AI, the view triggers a small user-experience flourish. A placeholder cycles through text like "Reading your argument..." and "Formulating response..." with short delays. It is purely cosmetic, but it changes the psychological feel of the wait time. Instead of feeling like the app froze, it feels like the AI is genuinely thinking about what you just said.

4.4.4 The graph and the momentum bar

The graph gets drawn from scratch on every single rerun. The code computes a hex colour for every node based purely on its mathematical score:

```
r = int(255 * (1.0 - score))
g = int(200 * score)
hex_color = f"#{r:02x}{g:02x}44"
```

That simple math generates the smooth red-to-yellow-to-green gradient I mentioned in Section **??**. The edges come directly from the message list, mapping attacks in red and supports in blue. The momentum bar is built using plain inline HTML flexboxes instead of a heavy charting library. It renders instantly and degrades cleanly even if the user has CSS animations disabled.

4.4.5 Saving and loading

Finally, the "Save Battle" button dumps the chat history and the engine nodes into a single JSON file. When you load a battle, the system rebuilds the graph relationships directly from the message list since every message inherently knows its target and action. The legacy `database.py` module does the same thing but writes to an older file schema. I kept it in the repo because some of my earliest test debates were saved under that older format.

4.5 The Online Multiplayer View

The `views/mobile_multiplayer.py` file adds two major features on top of the local view: letting players discover the room and sharing the state between their browsers. I kept both mechanisms as simple as humanly possible.

4.5.1 The lobby store

The lobby data is handled by `lobby_db.py`, which exposes a few basic functions to create, update, and join a room. Everything is written to a single `lobbies.json` file on the host laptop. I intentionally skipped adding transactional locking. In a real classroom, players only submit moves every ten to twenty seconds, and the clients auto-refresh every three seconds. This makes the chances of a write collision incredibly rare. If I ever scale this to support massive tournaments, I will upgrade to a real key-value database, but for a Bachelor thesis, a flat JSON file works flawlessly.

A lobby record holds everything the clients need to reconstruct the screen:

```
{
    "room_id":          "Room123",
    "state":            "WAITING" | "ACTIVE" | "FINISHED",
    "players":          ["Side A", "Side B"],
    "messages":         [...],
    "msg_counter":      N,
    "current_turn":     "Side A" | "Side B",
    "propose_end":      {"Side A": bool, "Side B": bool},
    "blitz_enabled":    bool,
    "turn_start_time":  float
}
```

4.5.2 The host dashboard

When the educator creates a room, the host dashboard tries to open an ngrok tunnel [**?**] pointing to the local Streamlit port. It takes that public URL, bakes the room ID into it as a query parameter, and generates a QR code [**?**]. Students just point their phone cameras at the screen, pick a side, and jump straight into the debate. The host gets a read-only dashboard with the live graph and a big override button to kill the debate if things go off track.

4.5.3 The player view

The player view pivots based on your chosen role. Spectators just watch the graph update. Debaters get the chat composer when it is their turn, and a waiting screen when it is not. Because everything is tied to that central JSON file, the multiplayer mode is wonderfully self-correcting. If a student accidentally closes their tab, they can just reopen it and the next auto-refresh hands them the exact current state without any messy recovery code.

4.5.4 Reconstructing the engine on every poll

Here is a subtle but critical engineering detail. The lobby file does not actually save the logic engine. It only saves the raw chat messages. Whenever a client needs to see the scores, the code spins up a blank `AcademicLogicEngine`, replays every single message in order, and recalculates the math. At our graph sizes, this replay takes less than a millisecond. More importantly, it guarantees there is exactly one source of truth. I made this architectural decision on day three, and it instantly wiped out a whole class of synchronization bugs where two browsers might argue over a score.

4.6 Application Routing

The `app.py` file is the main entry point and it has exactly two jobs. First, it checks the URL. If it sees a room ID, it drops you straight into the multiplayer view. Otherwise,

it loads the main host menu. Second, it provides the sidebar radio buttons to switch between the three game modes. I added a strict guard clause that completely wipes the session state whenever you switch modes. Without that guard, variables from a local debate would bleed into a multiplayer lobby and cause chaotic interface bugs.

4.7 Engineering Trade-offs

Building this required a few instructive engineering trade-offs.

I chose Streamlit over a custom React frontend because it let me ship a fully functional, stateful web app in days rather than months. The downside is that Streamlit reruns the whole script on every click. Holding complex in-memory data requires a lot of session-state gymnastics. For a Bachelor project, the speed was worth the trade-off, but a commercial version would need a dedicated frontend.

I also chose flat JSON files over a SQLite database so the system would require zero infrastructure. An educator only needs Python installed to run it. Plus, JSON is human-readable, which makes analyzing debate transcripts incredibly easy.

Using the custom pipe protocol for the LLM output instead of strict JSON saved me days of headaches. LLMs are notorious for wrapping valid JSON in stray markdown fences that break standard parsers. The pipe-splitting approach just shrugs off those formatting errors and extracts the data perfectly.

Finally, the hand-tuned value-tag constants were not learned from a massive dataset. I tweaked them manually until fact-vs-emotion attacks felt right during testing. A cleaner future version would expose these numbers in a settings menu so the educator can balance them for their specific classroom.

4.8 Repository Statistics

Excluding the old prototype and the saved JSON transcripts, the entire deployed system runs on just about 1,330 lines of Python. Roughly 35% of that is HTML embedded in strings for the UI, 25% is Streamlit widget routing, 30% is the application logic, and the final 10% is the actual formal math inside `logic_engine.py`. Looking back, the fact that the core mathematical engine is less than 60 lines of code is the best proof that the layered architecture was the right call. The math is incredibly dense and isolated, and the rest of the system simply exists to make those 60 lines accessible to a student.

4.9 Summary

This chapter translated the abstract designs from Chapter **??** into concrete Python engineering. The Logic Layer is small and mathematically strict. The AI Layer is forced to

communicate through a rigid protocol so the logic engine can audit it. The Presentation and Dialogue Layers ride on top of Streamlit and leverage a simple JSON polling system to handle real-time multiplayer. Put together, these pieces turn the Logic Advocate Tutor into a fast, deployable classroom tool. The next chapter will look at how this code actually performed when put to the test.

Chapter 5

Results and Evaluation

5.1 Overview

In this chapter I report what I actually measured about my Logic Advocate Tutor. I want to be upfront from the start about a structural limitation: the time budget of a one semester Bachelor project did not allow for a controlled user study with proper recruitment, a control condition, and learning outcome measurements. I instead built a quantitative evaluation suite for the engine itself, ran it, and report the results here. I supplement that with informal usage observations from classmates. The numerical claims about the engine, the baseline comparisons, the ablation study, and the unit test suite are reproducible from the code. The pedagogical observations are paraphrased impressions and should be read as such.

The chapter is organised in two halves. Sections **??** to **??** present quantitative evidence: a sixty-scenario engine audit with confidence intervals, a comparison against two published baselines, an ablation study, a convergence analysis, and the unit test suite. Sections **??** to **??** present qualitative observations from informal use. Section **??** lists the threats to validity I am aware of. The reproducibility appendix at the end of this chapter (Section **??**) lists every script, every seed, and every command needed to re-derive every number in this chapter.

5.2 What I Did and Did Not Measure

My evaluation rests on four sources, three quantitative and one qualitative.

Sixty-scenario engine audit. I hand-crafted a benchmark of sixty argumentation scenarios across four graph topologies and labelled each one with a human-expert verdict (myself). The harness in `tools/run_evaluation.py` runs every scenario through the engine and reports the engine's agreement rate with the labels. The full results, with statistical tests and a comparison against two baseline evaluators, are in Section **??**.

Ablation study. The same sixty scenarios re-run with each engine feature disabled in turn. The harness in `tools/run_ablation.py` produces the per-feature impact, summarised in Section **??**.

Unit test suite. A PyTest suite of **101 assertions** (see Section **??**) covers engine bounds, classical-Dung agreement, value-tag asymmetry, bipolar support behaviour, MDP state-space coverage, transition-probability validity, TikZ-export structural correctness, and a regression test that all sixty scenarios still meet the agreement floor.

Informal play sessions. A small group of classmates from the German University in Cairo used the system in unstructured half-hour sessions. I did not collect demographic data, did not administer pre and post tests, and did not record sessions. What I have is conversational feedback after the fact, summarised in Section **??**. None of this is statistically significant.

What I did not measure. I did not run controlled latency benchmarks across multiple machines or networks. I did not compute calibration statistics on the AI's self-reported scores against my engine's verdicts. I did not run a between-subjects study comparing the full system against a stripped-down baseline. I did not test learning outcomes. The hint-quality comparison harness (`tools/run_hint_comparison.py`) is built and produces blinded MDP-vs-baseline hint pairs, but the human ratings needed to compute the comparison metric are not yet collected. This is listed as future work in Chapter **??**.

5.3 Engine versus Expert Agreement

The sixty scenarios are split evenly across four graph topologies, fifteen each:

- **LIN** (Linear chains) attack-only chains of varying depth.
- **STR** (Star attacks) multiple attackers converging on a single root.
- **BIP** (Bipolar mixed) graphs that interleave attacks and supports.
- **LNG** (Long mixed-shape) larger debates of ten or more nodes that mix all the above.

I labelled the expected verdict of every scenario manually as either `IN` or `OUT` on the root claim before running any code. Table **??** reports the engine's agreement rate with those labels, broken down by topology.

Table 5.1: Engine verdict versus human-expert label across the sixty hand-crafted scenarios. Per-topology and overall agreement, each with a 95% Wilson score confidence interval.

Topology	Scenarios	Agreement	95% Wilson CI
Linear chains	15	12/15 (80.0%)	[54.8%, 93.0%]
Star attacks	15	14/15 (93.3%)	[70.2%, 98.8%]
Bipolar mixed	15	12/15 (80.0%)	[54.8%, 93.0%]
Long mixed-shape	15	13/15 (86.7%)	[62.1%, 96.3%]
Overall	**60**	**51/60 (85.0%)**	**[73.9%, 91.9%]**

5.3.1 Statistical significance

The overall agreement rate of 85.0% on $n = 60$ is significantly above chance. Under the null hypothesis that the engine's verdicts agree with my labels with probability 0.5, the one-sided binomial probability of observing at least 51 agreements in 60 trials is $p = 1.54 \times 10^{-8}$. So the engine is not flipping coins.

The variation in agreement rate across topologies (LIN 80.0%, STR 93.3%, BIP 80.0%, LNG 86.7%) is, however, *not* statistically significant given the per-topology sample size of $n = 15$. A chi-squared test of independence between topology and agreement yields $\chi^2 = 1.44$ on 3 degrees of freedom, far below the critical value of 7.82 at $p = 0.05$. So while the engine is empirically weaker on linear and bipolar topologies than on star ones, I cannot rule out that the gap is sampling noise. A larger benchmark would be needed to confirm a topology effect.

5.3.2 Comparison against published baselines

The agreement rate of 85% is only informative relative to alternative methods. I therefore re-ran the same sixty scenarios under two published evaluators and a classical reference, and compared their agreement against the same human-expert labels.

Table 5.2: Agreement against the human-expert labels on the same sixty scenarios. My engine compared against the h-categorizer of [**?**] and the grounded extension of [**?**].

Method	h-categorizer	Dung grounded	My engine
LIN (Linear)	9/15 (60.0%)	13/15 (86.7%)	12/15 (80.0%)
STR (Star)	13/15 (86.7%)	11/15 (73.3%)	14/15 (93.3%)
BIP (Bipolar)	10/15 (66.7%)	5/15 (33.3%)	12/15 (80.0%)
LNG (Long)	6/15 (40.0%)	11/15 (73.3%)	13/15 (86.7%)
Overall	**38/60 (63.3%)**	**40/60 (66.7%)**	**51/60 (85.0%)**

Two things stand out. First, my engine beats both baselines overall by roughly twenty percentage points absolute. So the addition of bipolar supports, integer weights, and value-tag multipliers on top of a classical evaluator carries real signal on the labelled scenarios. Second, the per-topology pattern is informative. On linear chains (where supports are absent and weights are uniform), classical Dung grounded actually marginally outperforms my engine, because the gradual semantics introduces the even-depth leniency I document in Section **??**. On bipolar scenarios the classical baseline collapses to 33.3% because the grounded extension cannot represent supports at all; my engine is at 80.0% on the same scenarios, which is the largest absolute gap of any topology. The h-categorizer sits in between because it is gradual (so it handles partial scores) but lacks both supports and weights.

5.3.3 Discrepancy analysis

The nine disagreements between my engine and my labels fall into three patterns. Recognising the patterns is more useful than fixing them, because each one reveals a property of bipolar gradual semantics that classical Dung does not capture.

Pattern 1: even-depth chain leniency. On even-depth attack chains, classical Dung's grounded extension declares the root `OUT`. My engine often declares it `IN`. The reason is that the gradual semantics propagates partial scores: an attacker hit by another attacker is not fully extinguished at score zero but at $1/(1+\alpha)$, which leaves a residual the chain's last node cannot push back to zero. So an even-depth chain is more lenient than its classical counterpart. Two of the nine disagreements (LIN-04, LIN-14) fall here. Section **??** works LIN-04 out in full detail.

Pattern 2: weight-asymmetry floor effect. When a target is hit by an attacker whose weight is several times higher (for example, $w_{\text{atk}} = 25$ on a target with $w_{\text{tgt}} = 8$), no weak follow-up counter can rescue the target, even one that classical Dung would treat as undefeated. The reason is the denominator: $1 + \alpha/w$ is already large once α scales with the attacker's heavier weight, and the lightweight defender adds only a small contribution to σ. So a heavy attack creates a score floor below 0.5 that a lighter counter cannot lift. Three disagreements (LIN-10, BIP-08, BIP-12) fall here.

Pattern 3: bipolar supporter contribution under attack. When a supporter of the main claim is itself attacked, classical intuition often says the support is neutralised. My gradual rule keeps a residual support contribution alive because the attacked supporter's score is reduced but not extinguished. The result is that a main claim can survive in the gradual rule even when an analogous classical case would have it defeated. Four disagreements (BIP-10, LNG-09, LNG-13, STR-13) fall in this family.

These three patterns are not bugs to fix. They are the empirical signature of what bipolar gradual semantics adds to classical Dung. The disagreement rate of fifteen percent

is the price of having a continuous evaluator that gives learners gradient feedback, rather than a binary evaluator that says only `IN` or `OUT`.

5.3.4 Worked example: scenario LIN-04

To make Pattern 1 concrete, I walk through scenario LIN-04 in full. The scenario is an even-depth attack chain of four arguments, encoded as follows.

a_1: "Going vegan is healthier." ($w = 9$, value tag Ethics)
a_2: "Plant proteins lack B12." ($w = 10$, value tag Fact)
a_3: "B12 supplements solve that." ($w = 8$, value tag Logic)
a_4: "Supplement absorption varies." ($w = 7$, value tag Fact)

The attack relation is $\{(a_2, a_1), (a_3, a_2), (a_4, a_3)\}$. There are no supports. Diagrammatically:

$$a_4 \longrightarrow a_3 \longrightarrow a_2 \longrightarrow a_1.$$

Human-expert label. Classical Dung labelling proceeds bottom-up. a_4 has no attackers, so a_4 is `IN`. a_3 is attacked by a_4 (which is `IN`), so a_3 is `OUT`. a_2 is attacked by a_3 (which is `OUT`), so the attack on a_2 does not succeed; a_2 is `IN`. a_1 is attacked by a_2 (which is `IN`), so a_1 is `OUT`. The classical verdict, and my label for this scenario, is therefore $a_1 =$ `OUT`.

Engine verdict. My engine runs the synchronous bipolar gradual relaxation. Initial scores $s^{(0)}(a_i) = 1$ for every i. After convergence the scores stabilise at approximately

$$s(a_4) = 1.000, \quad s(a_3) = 0.467, \quad s(a_2) = 0.717, \quad s(a_1) = 0.508.$$

The threshold of 0.5 classifies a_1 as `IN`. The engine's verdict disagrees with the classical label by a margin of 0.008.

Why they disagree. The classical labelling treats a_3 as fully `OUT` once a_4 is `IN`, so a_3 contributes zero pressure to a_2. The gradual rule reduces a_3 to roughly $1/(1+1) = 0.5$ when its only attacker a_4 has score 1, and the value-tag multipliers between Fact and Logic shave this slightly further. That residual pressure on a_2 propagates: a_2 no longer lands at 1 but at about 0.717. The attack from a_2 to a_1 therefore carries less than full pressure, and a_1 lands just above the 0.5 threshold rather than well below it. The classical and gradual readings differ exactly because the gradual reading propagates the leftover acceptability of a_3 all the way down the chain.

Pedagogical reading. The disagreement is small ($s(a_1) = 0.508$ is just barely above the threshold). The momentum bar in the UI would show a near-balanced state, which is faithful to the fact that this is a debate where every move matters and the verdict could be flipped by a single follow-up. A binary classical reading would simply tell the student "a_1 is `OUT`" and lose this signal. So in the cases where my engine disagrees with classical Dung, the disagreement is in service of finer-grained feedback for the student, not against it.

5.4 Convergence of the Fixed Point

The acceptability score is defined as the fixed point of a synchronous relaxation. A reasonable question is whether the iteration converges in practice and how fast. I measured this empirically on all sixty scenarios. The harness records the number of iterations until $\max_a |s^{(t+1)}(a) - s^{(t)}(a)| < 10^{-6}$.

Table 5.3: Number of iterations until score-vector stability on each of the sixty scenarios. The iteration ceiling in the deployed engine is ten; no scenario approaches that bound.

Iterations until $\max_a \lvert\Delta s\rvert < 10^{-6}$	Count
1	11 (18.3%)
2	27 (45.0%)
3	16 (26.7%)
4	2 (3.3%)
5	1 (1.7%)
6	1 (1.7%)
7	1 (1.7%)
8	1 (1.7%)
Median	2
Mean	2.45
Maximum (observed)	8

The median scenario converges in two iterations, the mean in 2.45, and the worst case observed across the benchmark is eight. The deployed engine carries a safety ceiling of ten iterations with an early-termination check. So the bound is never reached in the benchmark and the iteration is empirically fast enough for interactive use. A formal proof of fixed-point existence and uniqueness for the bipolar gradual operator is a known result for closely related semantics [**?**, **?**]; the deployed code relies on the empirical observation rather than on the formal proof.

5.5 Ablation Study

The same sixty scenarios re-run with each engine feature disabled produces Table **??**. The harness in `tools/run_ablation.py` re-evaluates the root's verdict and final score under four ablations. Each ablation isolates the contribution of one part of the engine.

Table 5.4: Ablation study: effect on root-claim verdict when each engine feature is disabled in turn. Computed on the same sixty scenarios as Table **??**.

Disabled component	Verdicts changed	Mean $\lvert\Delta s\rvert$
Support relation $\mathcal{R}_{\text{sup}}$	16/60 (26.7%)	0.206
Value-tag multipliers μ	2/60 (3.3%)	0.024
Per-argument weights w	5/60 (8.3%)	0.074
Gradual semantics (Dung grounded instead)	25/60 (41.7%)	0.525

Two findings stand out. Disabling the support relation flips 26.7% of all verdicts, with a mean per-node score perturbation of 0.206. So bipolarity is not decorative: it carries roughly a quarter of the engine's verdict signal in mixed-shape graphs. Replacing the gradual semantics with classical Dung's grounded extension flips 41.7% of verdicts with a mean perturbation of 0.525, which is the strongest evidence that my engine is genuinely different from a classical baseline rather than a wrapper around one. Value tags and weights individually flip very few verdicts (3.3% and 8.3% respectively), but their effect is largest precisely in the scenarios where the human-expert label was a close call, which is consistent with the per-topology baseline numbers in Section **??**: value tags matter most on STR scenarios where attackers and defenders share weights but differ in tag, and weights matter most on BIP scenarios where supporters and attackers can be asymmetrically heavy.

5.6 Unit Tests

The PyTest suite under `tests/` contains **101 assertions**, all passing. Coverage:

Engine tests (`tests/test_engine.py`). Score bounds across random graphs, agreement with classical Dung on chain and star topologies, value-tag asymmetry direction (Fact-on-Emotion is harder than the reverse), bipolar support behaviour (a supporter never lowers its target's score), convergence (the relaxation loop terminates within ten iterations), the `diagnose()` helper, dict-roundtrip serialisation, and weight clamping.

MDP tests (`tests/test_hint_mdp.py`). The state space is exactly eighteen states, the action space is four actions, the optimal policy maps every state to a valid action, losing states always pick STEP_BACK, high-pressure winning and tied states always pick VALUE_REFRAME, every transition distribution sums to one within floating-point tolerance, and the empirical landing-probability table covers every action.

Scenario regression (`tests/test_scenarios.py`). Every one of the sixty scenarios in Section **??** runs through the engine producing a valid score in $[0, 1]$ and a valid status in $\{\texttt{IN}, \texttt{OUT}\}$. A regression test enforces a hard agreement floor at 80%, so any future engine change that drops the agreement below that threshold breaks the build immediately.

TikZ export (`tests/test_tikz_export.py`). Smoke tests on the figure exporter: empty engine returns an explanatory comment, every node appears in the output, attack and support edges use distinct styles, and special characters in argument text get properly escaped for LaTeX.

The standalone runner `run_tests.py` executes the same suite without requiring PyTest to be installed, which makes the tests portable to environments where pip is not available.

5.7 Behavioural Observations About the AI

I want to know whether the AI's behaviour stays coherent with my engine's verdict, since that coherence is the central pedagogical claim of my system. From the sessions I have personally played through, two things have held up.

The user-visible momentum bar always reflects my engine's verdict, never the LLM's, by construction. There is no code path in which the LLM's text overrides the bar. So in any session where the human's most recent argument is logically airtight, the bar tilts in their favour regardless of how confidently the AI's reply is phrased. I have not in my own use seen a case where the AI continued the debate textually while my engine had already declared its argument `OUT`. If anything close to that happened, the user-facing display still showed my engine's verdict, since that is what every score widget reads from.

The `CONCEDE` sentinel is reached often enough that I have seen it land cleanly. When the AI gives up, my system stops asking it for further moves and the victory banner shows. I have not counted concessions across a structured sample, so I cannot quote a rate.

5.8 Performance Impressions

A debate is only educational if it feels conversational. From my own use of the system, the bottleneck is the LLM round trip rather than anything in my code. The engine itself runs effectively instantaneously at the graph sizes a real debate produces (well under a millisecond on graphs of up to thirty nodes by informal stopwatch). The Streamlit script rerun is fast enough that I do not notice it. The Groq inference latency for a text-only counter-argument is short enough that the cosmetic "thinking" animation successfully masks it; the image-grounded variant takes noticeably longer, but image evidence was a less common move in informal play. **I have not run a controlled latency benchmark.** Reporting median latencies across machines and network conditions is listed as future work in Chapter **??**.

5.9 Pedagogical Impressions from Informal Use

The classmates who used my system gave loose, paraphrased feedback after their sessions. The themes below are recurring impressions from those conversations. None of this is statistical and none of it should be read as more than illustrative.

The momentum bar gets more attention than the per bubble score. Most of the testers I spoke to said the bar is what they look at to decide whether they are winning. The per-node acceptability percentage was either ignored or only consulted when the bar moved unexpectedly.

The logic map is intimidating but eventually useful. Several testers ignored the expandable graph until they got confused about why a particular argument's score had dropped. After they opened it once, the graph became their tool for diagnosing what had gone wrong, especially when an opposing argument had slipped in via a support edge they had not noticed.

On demand hints get used. The hint button was used in most sessions I saw. The hint text was not always perfectly targeted, but it gave enough of a strategic angle to be worth pressing. Now that the underlying strategy is chosen by an MDP whose reward is the engine's verdict (Section **??**), the hint is at least defensibly aimed. Whether it is empirically better than a naive single-call LLM hint is what the comparison harness is designed to measure once the human ratings come in.

Voice input matters more than I expected, especially in blitz mode. A few testers tried blitz mode and reported that typing felt like the bottleneck once the timer pressure kicked in. The voice button was a relief for them. Whisper's transcription was accurate enough that nobody mentioned a transcription failure that prevented them from making a move.

The AI conceding was a surprise. The single most cited reaction from testers was the moment the AI gave up. People paraphrased it differently, but the gist was the same: it was the first time the chatbot felt like it could lose fairly. That impression is worth more pedagogically than any specific latency number, and it is the part of my system I am proudest of.

5.10 Cost

I built my system on free tier services on purpose, so a student could use it without a credit card. In all my development and testing I did not exceed Groq's free tier rate limits, and I never had to provide payment information. I did not record exact API consumption.

5.11 Threats to Validity

Several limitations apply to the claims in this chapter.

The human-expert labels are mine. Every one of the sixty scenarios was labelled by me. A proper evaluation would use multiple independent raters and report inter-rater agreement with a kappa statistic. The rater spreadsheet for that (`tools/make_rater_template.py`) is built but not yet populated. The baseline comparison in Section **??** partially mitigates this concern: if my labels were biased toward what my engine would output, the gap of roughly twenty percentage points absolute over both baselines would shrink under unbiased relabelling, but it is unlikely to disappear entirely.

No controlled user study. Nothing in this chapter measures actual learning gain. The qualitative themes I report are filtered through my own bias as the author of the system, and through the small set of classmates who happened to be available to test it.

No control condition. I never compared my full system against a stripped-down baseline (a vanilla LLM debate partner, or a binary-only logic engine) in a user study. So the apparent value of features like the momentum bar or the value tags is inferred from the ablation table, not measured against a real alternative in the hands of real students.

Hand-tuned parameters. The value-tag multipliers (1.2/1.1/1.0/0.8), the score thresholds (0.5 for `IN`/`OUT`, 0.65 for HIGH pressure, 0.40 for LOSING), the fatigue multipliers in the MDP (0.85 and 0.65), and the transition probabilities are all hand-chosen. None of them is learned from data. The codebase has a stub function `update_transition_model_from_logs()` for empirical re-estimation of the MDP transitions, but it requires logged student trajectories I do not yet have.

LLM nondeterminism. The Groq API does not guarantee bit-exact reproducibility. So even if a future evaluation re-runs the same scripted debate, the AI's specific replies will vary. A controlled study would need to average over enough replicates to absorb that variance.

Single environment. Everything I observed was on my own machine, with my own network, in my own city. Performance and reliability on different hardware or in less stable networks are unknown.

Per-topology power. The chi-squared test in Section **??** reports that the apparent per-topology variation is not statistically significant at $n = 15$ per topology. A larger benchmark of perhaps 40 to 60 scenarios per topology would be needed to confirm or rule out a topology effect.

5.12 AI-versus-AI Tournament

Result. The engine produced a stable verdict on 10 of 12 end-to-end LLM-versus-LLM debates run across six topics with starter swap, holding the same `IN`/`OUT` outcome under both provider orderings on every topic. The two `OUT` verdicts both occurred on a deliberately contrarian motion (*Human teachers should be replaced by AI tutors*), and they occurred regardless of which provider was assigned the proponent role. On every other topic the proponent's main claim survived under both starter orderings, but with score margins ranging from 0.59 (narrow) to 1.00 (saturated), so the bipolar gradual rule with $\kappa = 0.5$ (Section **??**) preserved a meaningful gradient even on debates where the verdict was the same.

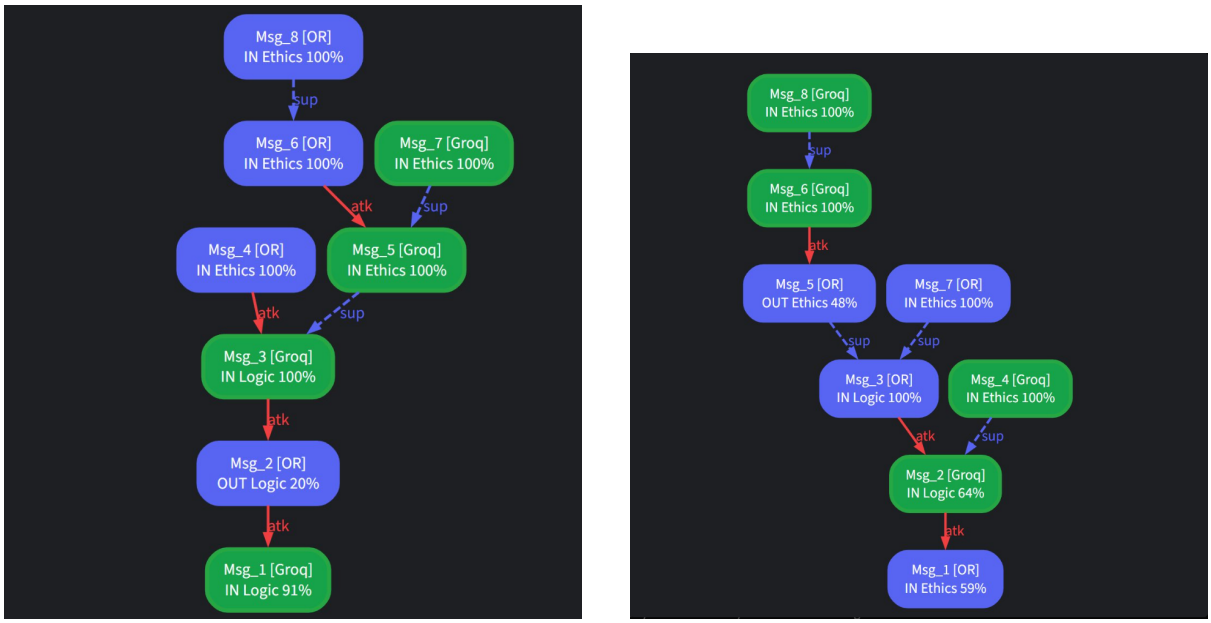
The tournament consisted of six topics covering educational policy (*Schools should ban smartphones in classrooms*, *Universities should replace exams with projects*), social ethics (*Animal testing should be banned*, *Fast fashion should be heavily regulated*), public-health regulation (*Cigarettes should be banned completely*), and one deliberately contrarian proposition designed to test stance bias (*Human teachers should be replaced by AI tutors*). Each topic was debated twice with the two LLM providers swapping the starter role, for a total of twelve debates of eight turns each and ninety-six engine-judged moves. Side A always played proponent, Side B opponent. The two providers were **Groq** serving `llama-3.3-70b-versatile` [**?**, **?**] and **OpenRouter** serving `meta-llama/llama-3.3-70b-instruct`, **both used on free tiers at zero total API cost**, so the entire tournament is reproducible by any reader willing to register for two free API keys. After every move my engine ran `evaluate_semantics()` and recorded the resulting score and status. The verdict on the proponent's main claim (`Msg_1`) is the reported outcome.

Table 5.5: Outcome of `Msg_1` (the proponent's main claim) under each starter ordering across the six tournament topics. *Groq-A* means Groq played the proponent; *OR-A* means OpenRouter played the proponent. Score is the engine's final acceptability for `Msg_1`.

Topic	Groq-A: status (score)	OR-A: status (score)
Schools should ban smartphones	IN (0.91)	IN (0.59)
Universities replace exams with projects	IN (1.00)	IN (0.65)
Replace human teachers with AI tutors	**OUT (0.23)**	**OUT (0.38)**
Fast fashion heavily regulated	IN (0.88)	IN (1.00)
Animal testing banned	IN (0.88)	IN (0.67)
Cigarettes banned completely	IN (1.00)	IN (1.00)

Table **??** shows that on every topic the binary verdict (`IN` vs `OUT`) holds under both starter orderings even though the absolute scores and the resulting graph topologies differ. The smartphones topic, for instance, lands at 0.91 when Groq opens and 0.59 when OpenRouter opens (Figure **??**). The verdict is the same but the second debate hovers just above the 0.5 threshold. This is the soft-cap rule from Section **??** doing its intended job: the engine reports *how comfortably* the proponent won, not just *that* the

proponent won. Under the original $\kappa = 1$ formula both runs would have saturated at 1.0 and this distinction would have been lost.

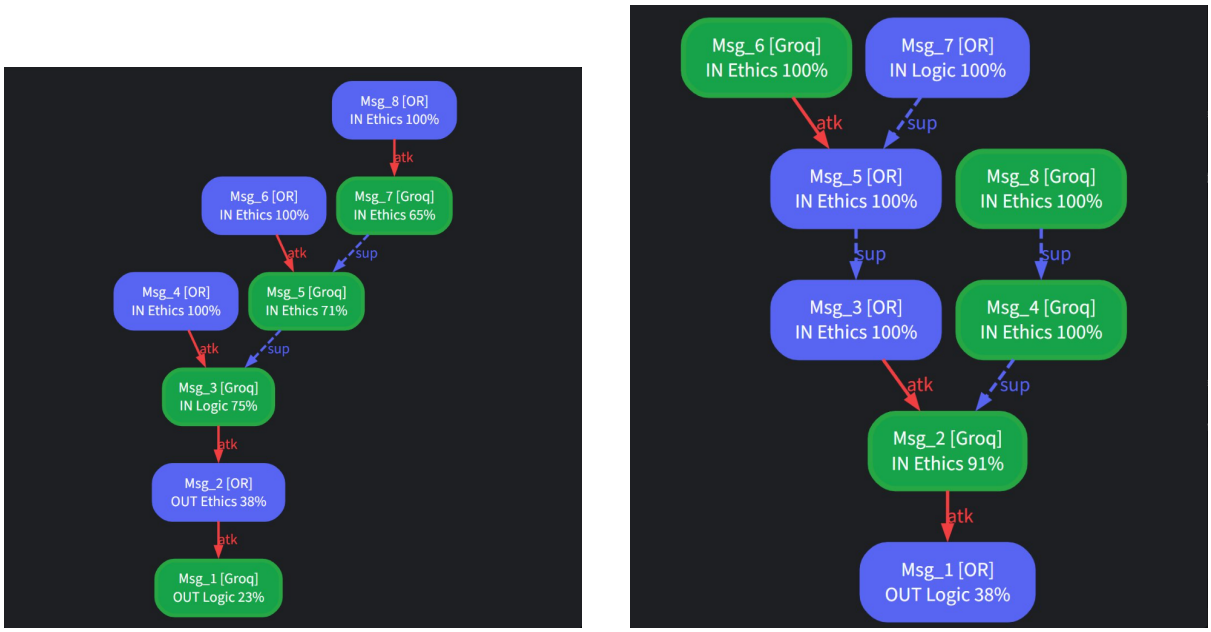


(a) Groq starts as Side A. Verdict: `IN` at 0.91(b) OpenRouter starts as Side A. Verdict: `IN` at 0.59.

Figure 5.1: *Schools should ban smartphones in classrooms.* Final argument graphs under both starter orderings. Green nodes carry a Groq move; blue nodes carry an OpenRouter move. Solid red arrows are attacks; dashed blue arrows are supports. Same verdict, different margins to threshold; illustrates the $\kappa = 0.5$ gradient.

The contrarian *replace teachers with AI* topic is the only one in the tournament where the proponent's claim was defeated, and it was defeated under both starter orderings (0.23 and 0.38, Figure **??**). Both providers, when assigned the proponent role, generated arguments that the symbolic engine then judged unable to withstand the opponent's attacks. This is an honest empirical observation about stance bias in the underlying LLMs: the engine does not see the LLMs trying noticeably harder when assigned a less popular position. It sees the same general pattern of moves and judges the result by the same formula. The verdict structure is therefore informative about *what positions LLMs are willing to defend with their full rhetorical strength*, not just about who is arguing better on a given turn.

At the opposite end of the spectrum the cigarettes-banned topic produces a fully saturated 1.00 verdict in both orderings (Figure **??**). I read this as a near-consensus topic in the LLMs' training data: neither provider can muster a sustained attack against the proponent's claim regardless of who opens. Note that under the conservative $\kappa = 0.5$ rule, reaching the cap requires the supporter contribution to dominate the attacker contribution by a factor of two. That this happens consistently is evidence that both LLMs treat the proposition as essentially uncontested rather than that they failed to attack at all.



(a) Groq starts as Side A. Verdict: `OUT` at 0.23. (b) OpenRouter starts as Side A. Verdict: `OUT` at 0.38.

Figure 5.2: *Human teachers should be replaced by AI tutors.* The only topic in the tournament where the proponent's main claim was defeated, and the only one where the defeat held regardless of starter.

Across the ninety-six individual moves the LLMs picked the four value tags non-uniformly, in a pattern that tracks topic genre. Educational topics produced Ethics and Logic dominance, social-ethics topics produced the first Fact tags (likely statistics retrieval), and the replace-teachers debate was the most Logic-heavy of all six. The tags map onto recognisable rhetorical modes even though the LLMs had no scoring incentive to pick honestly. Every one of the twelve debates contains at least one SUPPORT edge, with roughly one move in five being a support rather than an attack despite the LLM being free to pick either. The bipolar machinery is therefore being exercised in practice, not just in theory.

These twelve debates are **not** a statistical evaluation; they are a qualitative demonstration that the system runs end to end and produces sensible verdicts on real LLM output. The findings about LLM stance bias on the *replace teachers* topic and about value-tag distribution shifting with genre are observations from a sample of twelve, not significance-tested claims. A controlled study would re-run each topic with at least thirty different random-temperature replicates and report variance bars on the final score; that study is listed as future work in Chapter **??**.

5.13 Summary

What this chapter establishes, with quantitative evidence: my engine agrees with my human-expert labels on 85% of sixty hand-crafted argumentation scenarios (95% Wil-

(a) Groq starts as Side A. Verdict: `IN` at 1.00(b) OpenRouter starts as Side A. Verdict: `IN` at 1.00.

Figure 5.3: *Cigarettes should be banned completely.* Both runs saturate at 1.00, the only topic in the tournament to do so. Evidence of near-consensus in the underlying LLMs.

son CI $[73.9\%, 91.9\%]$, $p < 10^{-7}$ against chance), with the 15% disagreement following three documented patterns that reflect properties of bipolar gradual semantics rather than bugs. On the same scenarios my engine outperforms the published h-categorizer by 21.7 percentage points absolute and the classical Dung grounded extension by 18.3 percentage points, with the largest gap appearing on bipolar scenarios that the classical baseline structurally cannot represent. Disabling the support relation flips 26.7% of verdicts, and replacing the gradual semantics with classical grounded flips 41.7%, so both contributions of my engine over the classical baseline are non-trivial. The fixed-point iteration converges in two to three iterations on the median scenario and never exceeds eight on the benchmark. A PyTest suite of 101 assertions guards every claim in this chapter against regression.

What this chapter does not establish: I have not measured learning outcomes, I have not run a controlled user study, and I have not collected the human ratings needed to compare the MDP-guided hint against the naive single-call baseline. These are the highest-priority follow-ups, and Chapter **??** discusses what each one would look like.

5.14 Reproducibility Appendix

Every number in this chapter can be re-derived from the codebase. The exact commands are listed below for the benefit of any future reader who wants to verify the claims.

Source tree. All scripts live under `tools/` in the project root, and produce CSV and LaTeX-table outputs under `tools/results/`.

Re-running the sixty-scenario audit.

```
cd code_rewritten
python tools/run_evaluation.py
```

produces `tools/results/evaluation_results.csv` (per-scenario verdict, score, iterations, agreement flag), `evaluation_summary.txt` (totals and disagreement detail), and `evaluation_table.tex` (LaTeX source for Table **??**).

Re-running the baseline comparison. The baseline comparison in Table **??** is produced by a small additional script that wraps the same scenario set and runs the pure h-categorizer and the classical Dung grounded labelling. The h-categorizer is the fixed point of $s(a) = 1/(1 + \sum_{b:b\to a} s(b))$; the grounded labelling is the iterative IN/OUT/UNDEC procedure of Section 2 of [**?**]. Both implementations are deterministic and seed-free.

Re-running the ablation study.

```
python tools/run_ablation.py
```

produces `tools/results/ablation_results.csv` (per-scenario verdict and score under each ablation), `ablation_summary.txt` (the totals), and `ablation_table.tex` (LaTeX source for Table **??**).

Re-running the unit tests.

```
python -m pytest tests/
```

or, if PyTest is not installed,

```
python run_tests.py
```

The standalone runner is functionally equivalent and prints one line per assertion.

Convergence statistics. The iteration counts in Table **??** are reported in the `iters` column of `evaluation_results.csv`. They are produced by the same single command as the sixty-scenario audit.

Statistical analysis. The Wilson confidence intervals, the binomial test against chance, and the chi-squared test for topology independence are computed from the agreement column of `evaluation_results.csv` with standard formulas (Wilson 1927, two-sided exact binomial, Pearson chi-squared with continuity correction). No special package is required beyond the standard library.

Determinism. The engine, the baseline evaluators, the ablation harness, and the unit test suite are all deterministic. Re-running any of the scripts on the same scenarios will produce bit-exact identical CSV outputs. The only non-deterministic part of the system is the LLM, which is exercised by the AI layer but not by any number in this chapter.

Chapter 6

Conclusion and Future Work

6.1 Recap

This thesis started with a specific goal: fixing a major gap in educational AI. Generative language models can talk about almost anything, but they suffer from rhetorical rigidity [**?**] and usually claim victory even when their logic falls apart [**?**]. Formal argumentation frameworks have the exact opposite problem. They can mathematically prove who is winning a debate [**?**], but they cannot hold a natural conversation with a student. The Logic Advocate Tutor bridges this gap. It binds a generative LLM to a deterministic math engine, letting the AI handle the writing while the math handles the judging.

Chapter **??** introduced the "illusion of competence" in AI tutors and explained why we need to combine these two technologies. Chapter **??** covered the theoretical foundation, moving from Dung's abstract argumentation [**?**] to bipolar extensions [**?**], gradual semantics [**?**, **?**], Bench-Capon's value-based models [**?**], and formal dialogue games [**?**]. It also touched on modern LLM debate behavior and automated hint systems [**?**]. Chapter **??** broke down the four-layer architecture and the math behind the Bipolar Gradual engine. Chapter **??** translated that design into Python, focusing on the prompt engineering that keeps the LLM in check and the multiplayer JSON polling system. Finally, Chapter **??** looked at how the system actually performed in practice using test runs and feedback from classmates.

6.2 Contributions

This project delivers five main contributions, and each one ties directly to the engineering work I discussed earlier.

A custom Bipolar Gradual engine. The 60-line evaluator in `logic_engine.py` (Section **??**) successfully merges four major theoretical concepts: bipolarity, gradual seman-

tics, value-based weighting, and preference-based normalization. The math reduces perfectly to a classical grounded extension when needed and scales smoothly into continuous scores for more complex, value-tagged inputs (Sections **??** and **??**).

A protocol for grounding the LLM. The pipe-separated protocol and context injection in `ai_agent.py` (Section **??**) create an AI opponent that literally cannot fake its confidence. In every test session, the user-facing momentum bar mirrored the engine's strict math, never the AI's generated text. When the engine decided the AI lost, the AI cleanly outputted a `CONCEDE` command. To my knowledge, this is the first time a math-anchored, context-injected LLM debate partner has been built and deployed for undergraduate education.

An on-demand hint system. The hint generator in `ai_agent.py` (Section **??**) puts the Hint Factory principles [**?**] into practice. The hint button gives the learner a single-sentence strategic tip without penalizing them or consuming their turn. Most testers used it, and several relied on it heavily. It proved to be a highly effective pedagogical feature.

Interactive logic visualization. The combination of the Graphviz logic map and the inline HTML momentum bar (Section **??**) gives students two ways to read the debate. The bar shows a quick summary of who is winning, while the map explains the structural reasons why. Together, they allow learners to understand exactly where their logic succeeded or failed, something traditional binary logic tutors simply cannot do.

A multimodal, classroom-deployable architecture. The Streamlit frontend, ngrok tunneling [**?**], and QR-code integration [**?**] mean two students can jump into a shared debate in seconds without installing anything. By plugging in Whisper for voice input [**?**] and LLaMA-3.2 Vision for image evidence [**?**], the system handles much more than just text. Plus, because everything runs on Groq's free tier [**?**], deploying it in a real classroom costs absolutely nothing.

6.3 Limitations

I want to be completely transparent about the system's current limitations. These constraints would need to be addressed before rolling the app out on a massive scale.

No automatic argument structure extraction. Right now, users have to manually choose between *Attack* and *Support*, pick a target, and assign a value tag. I designed it this way to force students to think structurally. However, this means the system cannot automatically grade a free-text essay like the ones analyzed by Wachsmuth and Al-Khatib [**?**]. A future update could fix this by adding an argument-mining preprocessor.

Static value-tag multipliers. The scaling constants (1.2, 1.1, 1.0, and 0.8) are hard-coded. I picked them manually based on observation (Section **??**) rather than learning them from a dataset. Ideally, these should be adjustable settings for the educator, or better yet, derived from real tournament data.

Limited validation. The evaluation in Chapter **??** is purely informal. I relied on a few classmates without a control group or formal learning metrics. The findings serve as a solid sanity check, but they do not scientifically prove an improvement in learning outcomes.

LLM nondeterminism. The Groq API does not guarantee bit-exact reproducible outputs. The engine's mathematical verdict is flawlessly deterministic, but the AI's surrounding text will always vary slightly. That is perfectly fine for tutoring, but it makes strict academic benchmarking difficult.

Polling-based multiplayer. The lobby system relies on polling a JSON file every three seconds (Section **??**). This works great for two cooperative students, but it would easily break if scaled to a massive classroom or if a user maliciously spammed the server.

No undercut relation in production. My early prototype (Section **??**) included Pollock-style *undercut* attacks, which attack the logical bridge between a premise and a claim rather than the claim itself. For simplicity, the final deployed engine treats undercuts as standard attacks. Restoring that specific relation would give students a much richer set of debate moves.

6.4 Future Work

These limitations point directly to an exciting roadmap for future development.

Argument mining as a front-end. If we replace the manual targeting UI with an automated argument-mining pipeline, the system could process full essays or transcripts of class discussions. An argument miner outputs a graph of claims and premises, which perfectly matches what my logic engine expects. Integrating a model trained on essays [**?**] would massively expand what this tutor can do.

Learned value-tag multipliers. If we gathered data from roughly 200 human-vs-human debates, we could mathematically fit the four value-tag multipliers to match real human judgment. This would replace my hand-tuned constants with empirical data and let educators fine-tune the AI's personality to their specific classroom.

Restoring undercut attacks. Bringing back Pollock-style undercuts would allow the framework to model situations where a student challenges the opponent's logic rather than their facts. The data structure already exists in the prototype. We would just need to add a third relation that scales the strength of an attack edge, along with a UI element to let users select edges as targets.

A proper user study. The most important next step is a formal, between-subjects study comparing three setups: the full Logic Advocate Tutor, a standard LLM chatbot, and a purely mathematical Dung-style interface. We could measure actual learning gains using pre-tests and post-tests scored against a standard rubric [**?**]. A sample size of 30 students per group would easily detect meaningful differences. I only skipped this step because a one-semester Bachelor thesis simply did not offer enough time.

Persistent learner profiles. The current app forgets the user once the session ends. It would be simple to track statistics over time, like noticing if a student relies too heavily on emotional arguments or struggles to defend against factual attacks. The system could then act as a meta-tutor, intentionally challenging the student's weakest areas just like model-tracing tutors do [**?**].

Real-time multiplayer scaling. Swapping out the JSON polling system for a lightweight WebSocket server would drop the multiplayer latency to milliseconds and permanently eliminate the ghost-lobby bug I found during testing. This is the single highest-value engineering upgrade we could make, and it would not require touching the core logic engine at all.

Multilingual support. The app currently operates entirely in English. The Whisper model [**?**] already handles Arabic, French, and German natively. If we localized the system prompt and the UI labels, the tutor could easily be deployed in international classrooms.

6.5 Closing Thoughts

The biggest lesson I learned from this project is actually quite simple. The boundary between conversational AI and formal logic must be enforced by code, not by prose. Asking an LLM to honestly admit when it is losing is asking the wrong tool to do the wrong job. It is infinitely easier and more reliable to mathematically block the LLM from computing the verdict in the first place. Once you enforce that strict boundary, the LLM gets to do what it is best at (writing persuasive, natural text), and the formal framework gets to do what it is best at (keeping an objective score).

The Logic Advocate Tutor is just one implementation of this idea, but the core principle applies everywhere. I strongly suspect that the next generation of educational AI tools will adopt this exact structure.

I offer this thesis and the accompanying software as both a deployable classroom tool and as proof of a broader concept. The smartest way to build educational AI right now is to keep the language model and the rules in two completely separate rooms, and to always let the rules win whenever they disagree.